



How to Stop/Start/Restart Fabric Pods Including Rolling Restarts

Technical Note

Date: 30 January 2025

Version: 1.0

This document contains copyrighted work and proprietary information belonging to K2view. All intellectual property rights (including, without limitation, copyrights, trade secrets, trademarks, etc.) evidenced by or embodied in and/or attached, connected, or related to this document, as well as any information contained herein, are and shall be owned solely by K2view. K2view does not convey to you an interest in or to this document, the information contained herein, or its intellectual property rights, but only a personal, limited, fully revocable right to use the document solely for review purposes. Unless explicitly set forth otherwise, you may not reproduce by any means any document and/or copyright contained herein.

Information in this document is subject to change without notice. Corporate and individual names and data used in the examples herein are fictitious unless otherwise noted.

Copyright © 2025 K2view Ltd. / K2VIEW LLC. All rights reserved.

The following are trademarks of K2view: K2view logo, K2view's platform.

K2view reserves the right to update this list document from time to time.

Table of Contents

1.	Recommendations	3
2.	Description	3
2.1.	Using K2cloud Orchestrator (Recommended for Cloud-Managed Environments).....	3
2.2.	Using Kubernetes Commands.....	3
2.3.	Restarting from Within a Pod	4

1. Recommendations

- **K2cloud Orchestrator:** Use Pause and Resume for environments managed through K2cloud Orchestrator. This feature abstracts Kubernetes complexities and ensures proper sequencing, while also performing a rolling restart.
- **Kubernetes Rolling Restarts:** Use for environments that need minimal downtime or when incremental configuration changes are applied across multiple pods.
- **Kubernetes Full Restarts:** Use scale down and scale up operations for environments that require complete restarts.
- **Manual Fabric Commands:** Use for specific debugging scenarios when direct pod access is necessary.

2. Description

By leveraging these options, including rolling restarts, you can maintain high availability and ensure safe, efficient updates or restarts of Fabric services.

Several approaches exist for stopping, starting, and restarting services when managing Fabric pods in a Kubernetes environment or through K2cloud Orchestrator. Below are best practices tailored for different scenarios, including options for rolling restarts. Where applicable, K2cloud Orchestrator is emphasized.

2.1. Using K2cloud Orchestrator (Recommended for Cloud-Managed Environments)

If you're using K2cloud Orchestrator, the most straightforward way to manage Fabric pods is through the **Pause** and **Resume** actions:

- **Pause:** Gracefully stops the Fabric environment, ensuring all active processes are complete before shutting down.
- **Resume:** Restarts the environment, restoring all pods and services accordingly.

This user-friendly method eliminates the need for manual Kubernetes interactions and ensures proper environment sequencing.

2.2. Using Kubernetes Commands

For environments where direct Kubernetes interaction is required (e.g., custom setups or troubleshooting), you can manage pods manually with options for complete restarts, individual restarts, or rolling restarts.

Restarting Individual Pods:

To restart a single pod:

```
kubectl delete pod <pod-name> -n [namespace]
```

Kubernetes automatically recreates the pod based on its configuration in the stateful set.

Restarting the Entire Environment:

For a complete restart of the environment:

1. Scale down all pods:

```
kubectl scale statefulset <statefulset-name> --replicas=0 -n [namespace]
```

2. Scale back up to the desired number of replicas:

```
kubectl scale statefulset <statefulset-name> --replicas=<desired-replicas> -n  
[namespace]
```

This ensures all pods are stopped and restarted in sequence, maintaining configuration consistency.

Performing a Rolling Restart:

A **rolling restart** updates pods incrementally, ensuring minimal downtime. Use this approach when updates or restarts must be non-disruptive:

1. Patch the stateful set to trigger a rolling restart:

```
kubectl rollout restart statefulset <statefulset-name> -n [namespace]
```

2. Kubernetes will restart each pod one at a time, waiting for the previous pod to become ready before proceeding to the next. This ensures service availability throughout the process. You can monitor the progress of the rolling restart:

```
kubectl rollout status statefulset <statefulset-name> -n [namespace]
```

2.3. Restarting from Within a Pod

This method is useful for debugging or restarting specific services without affecting the entire cluster.

For manual restarts of Fabric services:

1. Connect to the pod:

```
kubectl exec -it <pod-name> -n [namespace] -- /bin/bash -l
```

2. Run the k2fabric restart command:

```
k2fabric restart
```